SMART WATER METER · FRONTEND HANDOFF

방울이 프론트엔드 연동 인수인계

복구된 React UI가 FastAPI, 모델 인식, 로컬 알림, Capacitor Android와 AWS 운영 환경에 연결되는 흐름을 한 문서로 정리했습니다.

대상 브랜치 · feature_app_merge

Android appId · com.jjang.banguli

운영 API · http://13.53.120.64:8000

1. 한눈에 보는 프론트 연결 구조

화면은 외부 기능을 직접 호출하지 않습니다. App.jsx가 앱 상태와 화면 전환을 조정하고, 전용 어댑터가 API·사진·알림·기간 집계의 경계를 담당합니다.

레이어	핵심 파일	책임
화면	Onboarding · HomeScreen · CameraFlow · LedgerScreen · ReportScreen · ShopScreen · SettingsScreen	사용자 입력과 상태 표시
앱 조정	App.jsx	화면 전환, localStorage, 지역 선택, 검침 완료, 알림 갱신, 초기화
연동 어댑터	api.js · camera.js · notifications.js · reportData.js · regions.js	외부 API·네이티브 기능과 화면 사이의 안정된 경계
실행 환경	Vite 브라우저 · Capacitor WebView · FastAPI · Android 플러그인	개발과 앱 실행 환경 제공

화면 컴포넌트 맵

화면	파일	연동 포인트
최초 실행·홈	Onboarding.jsx · HomeScreen.jsx	지역 저장, localStorage, 설정·검침 진입
사진 검침	CameraFlow.jsx	camera.js → /meter/scan → /meter/manual
가계부·리포트	LedgerScreen.jsx · ReportScreen.jsx	/history · /history/period · reportData.js
설정	SettingsScreen.jsx · ReplacementHistoryScreen.jsx	알림, /history/replacements, /device-data
상점	ShopScreen.jsx	로컬 코인·보유·장착·방 꾸미기 상태

2. 최초 실행과 지역 설정



지역 저장 실패 시 선택과 모달을 유지해 같은 값을 재시도합니다. localStorage의 앱 설정과 서버의 기기별 지역 설정을 분리합니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
App.jsx · SettingsScreen.jsx	banguli_app_v1 · device_id	온보딩과 지역 모달의 표시 여부 결정
api.js · regions.js	GET /regions · PUT /settings	영문 지역 코드를 저장한 뒤 한글 이름으로 표시

3. 사진 검침과 저장

01 · CAMERA FLOW

검침 시작

촬영 버튼이 camera.js의 getPhoto()를 호출합니다.

→

02 · CAMERA.JS

사진을 File로 통일

Android는 Camera URI를 변환하고 웹은 파일 선택을 사용합니다.

→

03 · API.JS

모델 인식 요청

multipart image로 POST /meter/scan을 호출합니다. 아직 저장하지 않습니다.

→

04 · CONFIRM / MANUAL

사용자 확정

성공은 인식값 수정·확인, 실패는 수동 입력으로 전환합니다.

→

05 · FASTAPI

검침 저장

POST /meter/manual로 사용량·요금·환경 영향을 응답받습니다.

→

06 · APP.JSX

완료 반영

lastReadingDate를 갱신하고 오늘 알림을 제외한 뒤 결과 화면을 표시합니다.

파일 선택 취소는 서버 오류가 아니므로 촬영 화면으로 돌아갑니다. 자동과 수동 입력은 같은 저장 API를 사용하고 input_mode만 구분합니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
CameraFlow.jsx · camera.js	Capacitor Camera · 웹 파일 선택	사진을 File 한 종류로 통일
api.js	POST /meter/scan · POST /meter/manual	인식은 저장하지 않고 사용자 확정 뒤에만 DB 저장
App.jsx · notifications.js	recorded_at · lastReadingDate	결과 화면 표시와 오늘 알림 제외

4. 감소 검침과 계량기 교체 기준점

01 · FASTAPI

일반 저장 거절

현재값이 이전값보다 작거나 같으면 쓰지 않고 422를 반환합니다.

→

02 · CAMERA FLOW

교체 가능성 안내

값 재확인과 계량기 교체 선택을 분리합니다.

→

03 · CONFIRM

기록 보존 확인

기존 이력은 남고 새 값이 계산 기준이 됨을 두 번 확인합니다.

→

04 · API.JS

기준점 저장

같은 제출값을 reset_baseline=true로 다시 보냅니다.

→

05 · HISTORY

계산 재개

교체 직전 감소 구간을 제외하고 다음 검침부터 사용량을 계산합니다.

교체 기록은 GET /history/replacements에서 최신순으로 조회합니다. 감소값을 일반 검침으로 저장하는 우회 경로는 없습니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
CameraFlow.jsx · api.js	422 · reset_baseline=true	명시적 확인 전에는 감소값을 저장하지 않음
ReplacementHistoryScreen.jsx	GET /history/replacements	교체일·이전값·새 기준값을 최신순 표시
history.py	Reading.is_baseline	교체 직전 구간 제외 후 다음 구간부터 계산

5. 이력과 에너지 리포트

01 · LEDGER / REPORT

화면 진입

가계부와 리포트가 같은 device_id의 서버 이력을 요청합니다.

→

02 · API.JS

실제 이력 조회

GET /history 또는 날짜 범위의 /history/period를 호출합니다.

→

03 · REPORTDATA.JS

기간 구성

주간 7일, 월간 30일, 연간 365일과 비교 기간을 계산합니다.

→

04 · REPORT SCREEN

기간 비교

이번·이전 날짜와 사용량, 요금, CO2를 함께 표시합니다.

→

05 · UI STATE

예외 상태

빈 배열은 검침 CTA, 통신 실패는 오류와 재시도로 구분합니다.

날짜 지정은 현재 기간과 바로 이전의 같은 길이 기간을 각각 조회합니다. 선택 범위 안에 완전히 포함된 이력 구간만 합산합니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
LedgerScreen.jsx	GET /history	최신순 구간·빈 상태·검침 CTA 표시
ReportScreen.jsx · reportData.js	GET /history · GET /history/period	이번·이전 날짜 범위와 사용량·요금·CO2 비교
api.js	start/end=YYYY-MM-DD	네트워크 오류와 빈 배열을 서로 다른 UI 상태로 전달

6. 알림·설정·전체 초기화

01 · SETTINGS

알림 설정

사용 여부와 시각을 선택합니다. 기본 시각은 20:00입니다.

→

02 · NOTIFICATIONS.JS

60일 예약

권한을 확인하고 날짜별 일회성 알림 60개를 예약합니다.

→

03 · READING SUCCESS

오늘 건너뛰기

검침 완료 날짜를 반영해 오늘 알림을 제외합니다.

→

04 · SETTINGS API

교체 이력

GET /history/replacements로 현재 기기의 기준점을 표시합니다.

→

05 · RESET

서버 우선 초기화

DELETE /device-data 성공 뒤 알림과 localStorage를 삭제합니다.

서버 삭제 실패 시 로컬 상태를 보존합니다. 웹 알림 어댑터는 플러그인을 호출하지 않는 성공 no-op이고, Android 알림 탭은 검침 화면으로 이동합니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
SettingsScreen.jsx · notifications.js	Local Notifications	권한과 60일 예약을 직렬화해 교체
App.jsx	lastReadingDate	검침 성공일의 알림을 건너뜀
api.js	GET /history/replacements · DELETE /device-data	교체 이력 조회와 서버 우선 전체 초기화

7. 운영 빌드·APK·AWS 연결

01 · VITE

운영 번들

npm run build가 Elastic IP를 포함한 dist를 생성합니다.

→

02 · CAPACITOR

Android 동기화

웹 자산과 Camera·Local Notifications 플러그인을 복사합니다.

→

03 · GRADLE

Debug APK

JDK 21, SDK 36으로 app-debug.apk를 만듭니다.

→

04 · ELASTIC IP

운영 API

앱은 HTTP 13.53.120.64:8000으로 요청합니다.

→

05 · EC2 DOCKER

백엔드 운영

water-api가 weights·data 볼륨과 unless-stopped 정책으로 실행됩니다.

프론트와 APK는 EC2 Docker에 포함하지 않습니다. 현재 HTTP 연결 때문에 Android cleartext를 허용하며, 스토어 배포 전 HTTPS 전환이 필요합니다.

연결 파일과 데이터

관련 파일	API·플러그인·상태	팀원이 기억할 결과
.env.production · api.js	http://13.53.120.64:8000	운영 번들에 고정 Elastic IP 반영
capacitor.config.json · android/	Camera · Local Notifications · cleartext	Vite dist와 네이티브 플러그인을 APK로 패키징
Dockerfile · deployment-guide.md	water-api · weights/data 볼륨	API 자동 복구와 모델·SQLite 영속화

8. 실행·빌드·운영 명령

백엔드 로컬 실행

```
uv sync
uv run uvicorn src.api.main:app --reload --host 0.0.0.0 --port 8000
```

프론트 개발과 테스트

```
cd frontend
npm install
npm run dev
node --test
npm run build
```

Capacitor Android Debug APK

```
cd frontend
npx cap sync android
$env:JAVA_HOME = 'C:₩Program Files₩Android₩Android Studio₩jbr'
$env:ANDROID_HOME = "$env:LOCALAPPDATA₩Android₩Sdk"
cd android
.₩gradlew.bat assembleDebug
```

EC2 Docker 상태와 헬스체크

```
docker ps --filter name=water-api
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' water-api
curl -fsS http://localhost:8000/health
curl -fsS http://13.53.120.64:8000/health
```

9. 검증 결과와 머지 전 체크

영역	확인 결과	상태
Python	pytest 42건 통과. 기존 Starlette/httpx2 전환 경고 1건	통과
프론트	Node 계약·통합 테스트 59건 통과	통과
Vite	운영 번들 생성. 기존 500 kB 청크 경고	통과
Capacitor	Camera 8.2.1 · Local Notifications 8.0.0 · Android 8.4.1 동기화	통과
Android	Debug APK 빌드와 appId com.jjang.banguli 확인	통과
에뮬레이터	신규 코인 0, 온보딩, 서울 운영 API 저장과 홈 진입	통과
Android 실기기	실제 카메라와 로컬 알림 수신·탭 이동	미검증

남은 필수 확인

순서	실기기 E2E 항목
1	시스템 카메라 권한과 실제 계량기 사진 촬영
2	모델 인식 결과 확인, 수동 입력 폴백과 저장
3	알림 권한 허용·거부, 지정 시각 수신과 알림 탭 후 검침 화면 이동
4	앱 재실행 후 같은 device_id와 검침 이력 유지
5	실제 iPhone 사진의 HEIC 업로드 호환성

실기기 검증 전에는 Task 9를 완료로 표시하거나 APK를 정식 배포본으로 부르지 않습니다. dev 머지는 사용자와 팀의 승인 후에만 수행합니다.